



Discrete cosine transform for filter pruning

Yaosen Chen^{1,3} · Renshuang Zhou^{1,3} · Bing Guo¹ · Yan Shen² · Wei Wang^{1,3,4} · Xuming Wen^{3,4} · Xinhua Suo¹

Accepted: 10 March 2022 / Published online: 30 May 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Neural network filter pruning has demonstrated its effectiveness for deploying the models with fewer resources and efficient inference. However, the process of pruning networks in existing methods is complex and inefficient. This paper use Discrete Cosine Transform (DCT) to transform the feature map to the frequency domain and propose a simple and effective filter importance calculation method for filter pruning called DCTPruning. The important information of an image is usually contained in the low-frequency part. Discrete cosine transform transforms the image from the spatial domain to the frequency domain. The high-frequency part can be removed and lossy compressed without affecting the storage of important information. This study finds that this phenomenon is also applicable to the feature map of neural networks. Based on the discrete cosine transform, this study proposes a discrete cosine transform pruning method. A discrete cosine transform is used to calculate the importance of each filter in the neural network feature map, and the filter is pruned according to its importance. The proposed method not only achieved a good result in the classification task but also in the saliency object detection task. For the classification task, compared with the existing state-of-the-art, the proposed method has a significant improvement in terms of FLOPs and parameter reduction and maintains the accuracies. For example, for VGG-16 on CIFAR-10, this study achieves the parameters are reduced by 94.2% and FLOPs are reduced by 84.1%, the accuracy is only reduced by 1.43% (92.53% vs 93.96%); for ResNet-50 on ImageNet, DCTPruning provide the FLOPs are reduced by 74.1% and parameters are reduced by 70.8%, and the Top-1 accuracy is only reduced by 3.84% (72.31% vs 76.15%), the Top-5 accuracy is only reduced by 2.10% (90.77% vs 92.87%). For the saliency object detection task, this study also performs effective network pruning and achieves great model size reduction while keeping a similar performance.

Keywords Neural network · Filter pruning · Discrete cosine transform · Frequency domain

1 Introduction

With the remarkable progress of convolutional neural networks has been made in more and more computer vision tasks [28–31, 56–58], the deployment required more efficiency.

It required the models to keep the accuracy, while smaller model size, faster processing, and less computation. Especially with the development of the Internet of things [59, 60], more and more models need to be deployed at the edge. Chen et al. [53] discussed the importance of deep learning model compression in the stage of transition from big data to extensive knowledge. Due to computing resources and power consumption limitations, the edge end usually expects higher accuracy and efficiency. However, efficiency and precision are contradictory. Therefore, it is a great challenge to model compression.

The existing model compression techniques mainly focus on quantization and pruning. The quantification work is to replace the parameters in the original model with a smaller storage size to reduce the model size and increase the processing speed. However, quantization is limited. For example, quantization with smaller bits may require special hardware. Besides, quantization cannot change the structure of the network. It is found that the neural network has

✉ Bing Guo
guobing@scu.edu.cn

Yaosen Chen
chenyaosen@stu.scu.edu.cn

¹ College of Computer Science, Sichuan University, Chengdu, Sichuan, 610065, China

² School of Computer Science, Chengdu University of Information Technology, Chengdu, Sichuan, 610225, China

³ Media Intelligence Laboratory, ChengDu Sobey Digital Technology Co., Ltd, Chengdu, Sichuan, 610041, China

⁴ Peng Cheng Laboratory, Shenzhen 518055, China

enormous redundancy, most models can reduce some connections, but the effect of the model will not be significantly affected. Therefore, the proposed method can achieve the compression model's purpose by pruning and reducing the calculation and power consumption requirements.

The basic idea of pruning is to achieve network compression by removing the least important part of the network and then restoring the accuracy of the network by fine-tuning or other methods. Therefore, measuring the importance of the filter has become the key to pruning. One of the most straightforward heuristics is to evaluate the importance according to the absolute value [1] of the parameter or feature output. This kind of algorithm is called "magnitude-based weight pruning [6, 7, 16, 32]".

The above method assumes that the smaller the absolute value of the parameter, the less influence it has on the final result. However, this assumption may not be true [35]. The second way is to consider the influence of parameter clipping on loss. For example, OBD [13] and OBS [14] methods are discussed this way. However, the two methods are time-consuming due to the need to calculate the Hessian matrix or approximate comparison. In recent years, some researchers proposed to solve the problem of time-consuming [33, 34]. For example, Snip [33] pruning volatile neural networks for resource-efficient transfer learning is based on Taylor expansion but uses the absolute value of the first-order term in the expansion of the objective function relative to activation as the pruning criteria. In this way, the second-order term (Hessian matrix) calculation is avoided. However, the important calculation method and time consumption are still difficult challenges in network pruning. HRank [5] uses the filter pruning method by exploring the High Rank of feature maps (HRank), but it is limited to networks equipped with activation function (refer to 4.3.3).

Inspired by Discrete Cosine Transform (DCT) [17] in image compression [18, 19], this study proposes DCT transform the calculation method of importance from spatial domain to frequency domain as a solution to address this problem. This paper presents a simple and effective filter importance calculation method based on DCT for filter pruning called DCTPruning. As shown in Fig. 1, when pruning the convolution operation of a certain layer, firstly perform DCT transformation on the feature map of the layer; secondly, calculate the importance score of the filters in this layer, and use the input of multiple images to calculate the average importance score as the final importance score; then remove the filters of unimportant parts; finally, initialize the reserved network parameters. The pruning network is fine-tuned to reduce the size and complexity of the model while maintaining accuracy of the model. Although Liu et al. [51] have used DCT for model compression, its method is highly complicated and requires

a long time for dynamic pruning. It uses DCT to transform the feature map, simpler than the method proposed by Liu et al. [51] which needs to transform both the input and the weights. The designed method is a kind of static pruning, which only needs to be done in one-shot according to the importance score. The proposed method has more advantages in simplicity and efficiency, only involving DCT on feature maps. As a channel pruning method, it also has no specific model structures or training from scratch. And also, the proposed method can be applied in any form of convolutional neural network structure. The proposed method has achieved good results in classification tasks and the task of saliency object detection. Overall, this study has made the following contributions:

- This study proposes a simple and effective filter importance calculation method based on Discrete Cosine Transform for filter pruning, called DCTPruning.
- The proposed filter importance calculation can be applied in any form of convolutional neural network structure to achieve a wide range of channel importance assessments.
- The proposed method has a significant improvement in terms of FLOPs and parameters reduction. Furthermore, it maintains accuracy compared with existing state-of-the-art in the classification task. The proposed method is also effective in saliency object detection and has achieved good results.

The rest of this paper is organized as follows. Section 2 will introduce the work related to this paper. Section 3 will introduce the proposed approach for filter pruning in detail. Section 4 shows and analyzes the comparison results and the ablation experiment. Finally, conclusions are drawn in Section 5.

2 Related work

Model compression: Model compression aims to reduce the size of the model as much as possible while maintaining the similarity of the prediction accuracy effect. Methods of model compression include weight quantization, network pruning, neural architecture search, and knowledge distillation. Neural architecture search [24, 25] is an idea of automatically searching for neural network architecture, which can set with some restrictions to find more efficient models, but it needs huge computing resources. A scalable representation was proposed [52] the network parameters with hierarchically quantize the weights, so that can select the most suitable bit rate of the network based on their own storage constraints. Knowledge distillation [26, 27] can transfer the knowledge of a complex teacher network to

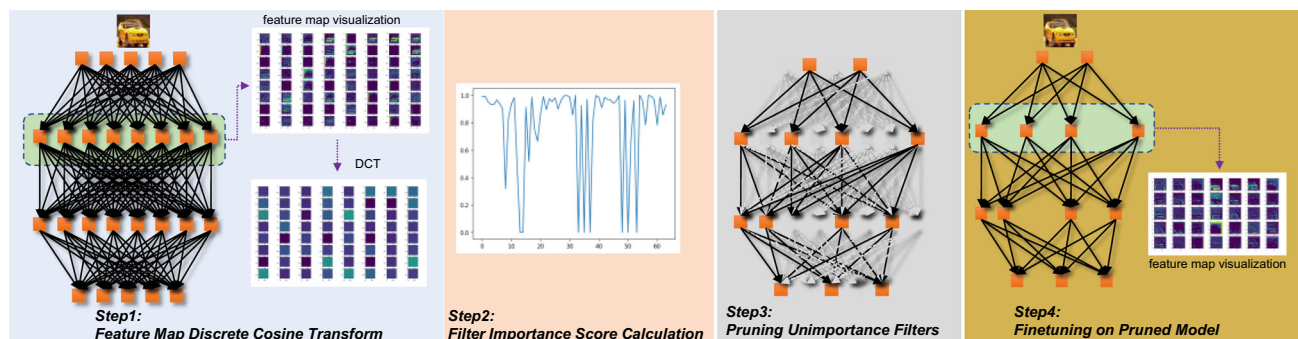


Fig. 1 The framework of DCTPruning. Step1, firstly perform DCT on the feature map of the layer; step2, calculate the importance score of the filters in this layer, and use the input of multiple images to

calculate the average importance score as the channel importance score; step3, remove the filters of unimportant parts; step4, fine-tuning on the pruned network

a simple student network, however, how to design a student network is still one of the keys. In terms of weight quantization, HashNet [21] establishes the mapping relationship through Hash and realizes the random sharing of weights, thereby reducing memory usage. In this method, only the shared weights and hash indexes need to be stored, thus significantly saving storage consumption, but it does not substantially help the acceleration of the model. To achieve model acceleration, binary neural networks appeared. This kind of network quantizes the weight and activation of the model into binary values. Courbariaux et al. [23] proposed a method to convert weights into binary values (-1, +1), which dramatically saves model storage consumption, but at the same time, it also produces a significant decrease in accuracy. In its method, Han et al. [22] uses a combination of network pruning, model quantization, and Huffman coding to compress the model. Dai et al. [60] proposed a tucker decomposition based on adaptive genetic method to compress the deep models. These studies, combined with a variety of compression methods, especially network pruning, can reduce the size of the model while maintaining similar accuracy.

Network pruning : Network pruning is a method to solve the over-parameterization [12] of deep neural network models. The purpose of network pruning is to reduce the model size and improve model performance. Network pruning is divided into two types: non-structured pruning and structured pruning. Early pruning work mainly focused on non-structured pruning. For example, the OBD and OBS methods proposed in Optimal brain damage [13] and SOD [14] respectively used the second-order derivative of the relative weights of the loss function to determine the redundant weights. Han et al. [15] proposed a method to determine the importance of weights by using the absolute value of weights and achieve the purpose of model compression by removing small weights.

However, since the kernel obtained by the non-structured pruning method is sparse unless the software and hardware have better support for sparse matrix operations, the pruned model will hardly have a speed increase. The savings in running memory are also limited. In this situation, researchers in recent years have focused on structured pruning. Structured pruning is different from non-structured pruning. Because it does not involve sparse matrix operations, structured pruning has a great effect on reducing the model size and speeding up inference. For example, using group LASSO [16] to obtain structured sparse weights to can achieve network pruning; utilize L1 regularization [7] with scaling factors of the BN layer and judges the importance of the channel by the value of the scaling factors to achieve network pruning. Lin et al. [5] found that the average rank of the feature maps output by one filter is always the same. Bilal et al. [54] combined L1-norm with capped L1-norm to represent the amount of information extracted by the filter and control regularization.

Discrete cosine transform (DCT): Discrete cosine transform [17] is widely used in image and audio compression. The signal or image is converted from the spatial domain to the frequency domain through the discrete cosine transform. Since the noise is concentrated in the high-frequency part, the purpose of noise reduction and compression can be achieved by removing the high-frequency signal. Although DCT is highly related to Discrete Fourier Transform (DFT), the weight of most images is usually concentrated in a few low-frequency components of DCT due to its spectral compaction, and DCT is a real-valued representation, so that DCT is used as a part of image compression in format JPEG [18, 19]. FreshNets [20] use a discrete cosine transform to convert the filter weights to the frequency domain and used a hash function to store the frequency parameters in groups to achieve the purpose of compressing the weight storage consumption. For an input matrix $M \in \mathbb{R}^{H \times W}$ and the

output matrix $\mathbf{D} \in \mathbb{R}^{H \times W}$ in frequency domain with DCT is defined as below:

$$D_{uv} = \frac{2}{\sqrt{HW}} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \cos \frac{(2x+1)u\pi}{2W} \cos \frac{(2y+1)v\pi}{2H} M_{xy} C(u) C(v) \quad (1)$$

where $x, u \in \{0, 1, 2, \dots, W-1\}$ and $y, v \in \{0, 1, 2, \dots, H-1\}$; $C(u) = \frac{1}{\sqrt{2}}$ in case $u = 0$, otherwise, $C(u) = 1$; and $C(v) = \frac{1}{\sqrt{2}}$ in case $v = 0$, otherwise, $C(v) = 1$.

Here, use $\mathbf{D} = \text{DCT}(\mathbf{M})$ to denote the DCT operation.

3 The proposed approach

In this Section 3.1 will introduce notation and preliminaries. Then, Section 3.2 will introduce the discrete cosine transform on feature maps. After that, Section 3.3 will introduce how to calculate the channel importance with discrete cosine transform. Lastly, Section 3.4 will introduce the pruning procedure with the discrete cosine transform method in this paper proposed.

3.1 Notation and preliminaries

In the convolution model of two-dimensional image processing, define Conv_i as the i -th convolutional layer operation:

$$\mathbf{F}_i = \text{Conv}_i(\mathbf{I}, :, :) \quad (2)$$

$\mathbf{F}_i = \{f_1^i, \dots, f_{C_i}^i, \dots, f_{C_i}^i\} \in \mathbb{R}^{C_i \times H_i \times W_i}$ is the output feature map of Conv_i , where C_i , H_i , W_i are the feature maps's channel size, height and weight, respectively. For each of the filter in $\mathbf{F}_i$ can be represented as $f_t^i \in \mathbb{R}^{H_i \times W_i}$, where $1 \leq t \leq C_i$. And $\mathbf{F}_{i-1} \in \mathbb{R}^{C_{i-1} \times H_{i-1} \times W_{i-1}}$ is the input feature map of Conv_i . For simplicity, ignore the non-linear operations include pooling, batch normalization, activation function and so on. Therefore convolutional filters in Conv_i can be defined as $\mathbf{W}_i = \{w_1^i, \dots, w_{C_i}^i, \dots, w_{C_i}^i\} \in \mathbb{R}^{C_{i-1} \times C_i \times k_i \times k_i}$, where k_i is the kernel size in Conv_i . For each of the filter in $\mathbf{W}_i$ can be represented as $w_t^i \in \mathbb{R}^{C_{i-1} \times k_i \times k_i}$, where $1 \leq t \leq C_i$. The purpose of filter pruning is to divide these filters into two parts according to their importance: $\mathbf{W}_i = \mathbf{W}_i^I + \mathbf{W}_i^U$, and then remove the unimportant part while keep the new pruned network keep the performance as the original one to achieve the goal of reducing the model size and computation. Where $\mathbf{W}_i^I$ is the group of importance filters with a quantity of C_i^I and $\mathbf{W}_i^U$ is the group of unimportance filters with a quantity of C_i^U . Hence, $C_i^I + C_i^U = C_i$.

3.2 Discrete cosine transform on feature maps

In the proposed method, describe the filter pruning as the following optimization problem:

$$\begin{aligned} \min_{\beta_{it}} & \sum_{i=1}^K \sum_{t=1}^{C_i} \beta_{it} \mathbb{E}_{I \sim P(I)} [\hat{\Omega}(\text{Conv}_{it}(\mathbf{I}, :, :))] \\ \text{subject to} & \sum_{t=1}^{C_i} \beta_{it} = C_i^2 \end{aligned} \quad (3)$$

Where K denotes that there are K convolution layers in the CNN model; β_{it} can indicate which group the filter should belong to, e.g., $\beta_{it} = 1$, the filter should be in $\mathbf{W}_i^I$ and $\beta_{it} = 0$, the filter should be in $\mathbf{W}_i^U$; $\mathbf{I}$ is the input image, which is a sample of the distribution of $\mathbf{P}(\mathbf{I})$, and $\hat{\Omega}(\cdot)$ is used to estimate the value of w_t^i generated feature map $\text{Conv}_{it}(\mathbf{I}, :, :)$. Each feature map, even in the same layer, plays a different role in the network. Therefore, the feature map can not only reflect the filter performance but also reflect the image characteristics. In addition, they can capture how the input image is converted to each layer up to the predicted tag layer. Similar to HRank [5], transfer this problem to how to design a suitable function $\hat{\Omega}(\cdot)$ to reflect the information richness of feature mapping. Unlike HRank, which uses Rank to define information measurement, use DCT to calculate the importance of the channel.

Hence, define the proposed information measurement as below:

$$\hat{\Omega}(\text{Conv}_i(\mathbf{I}, :, :)) = \text{DCTScore}(\text{Conv}_i(\mathbf{I}, :, :)) \quad (4)$$

Define $\mathbf{D}_i = \{d_1^i, d_2^i, \dots, d_{C_i}^i, \dots, d_{C_i}^i\} \in \mathbb{R}^{C_i \times H_i \times W_i}$ as the DCT map of the i -th convolutional layer's feature map $\mathbf{F}_i$. For each of the DCT output in $\mathbf{D}_i$ can be represented as:

$$d_{t,(x,y)}^i = \text{DCT}(f_{t,(u,v)}^i) \in \mathbb{R}^{H_i \times W_i} \quad (5)$$

where $1 \leq t \leq C_i$, $x, u \in \{0, 1, 2, \dots, W_i - 1\}$ and $y, v \in \{0, 1, 2, \dots, H_i - 1\}$.

In (1), when variables H and W are certain, $\frac{2}{\sqrt{HW}}$ is certain. Regard $C(u)$ and $C(v)$ in (1) as constant values, thus can ignore $\frac{2}{\sqrt{HW}}$, $C(u)$ and $C(v)$ in (1). Thus, rewritten (5) as:

$$d_{t,(x,y)}^i = \sum_{y=0}^{H_i-1} \sum_{x=0}^{W_i-1} \cos \frac{(2x+1)u\pi}{2W_i} \cos \frac{(2y+1)v\pi}{2H_i} f_{t,(u,v)}^i \quad (6)$$

Correspondingly, the inverse DCT can be written as:

$$f_{t,(x,y)}^i = \sum_{y=0}^{H_i-1} \sum_{x=0}^{W_i-1} \cos \frac{(2x+1)u\pi}{2W_i} \cos \frac{(2y+1)v\pi}{2H_i} d_{t,(u,v)}^i \quad (7)$$

where $f_{t,(x,y)}^i$ is the feature map in spatial domain and $d_{t,(x,y)}^i$ is the DCT feature map in frequency domain.

3.3 Channel importance caculation

Caculate channel importance score on the DCT feature map in frequency domain. First, caculate the sum of squares of all points on the DCT feature map:

$$z_t^i = \sum_{x=0}^{H_i-1} \sum_{y=0}^{W_i-1} (d_{t,(x,y)}^i)^2 \quad (8)$$

z_t^i is the element in the set of $\mathbf{Z}_i = \{z_1^i, \dots, z_t^i, \dots, z_{c_i}^i\}$, where $1 \leq t \leq c_i$. Then the channel importance score can be caculated by normalization as below:

$$\begin{aligned} S_i &= \{s_1^i, \dots, s_t^i, \dots, s_{c_i}^i\} \\ &= \frac{\mathbf{Z}_i - \min(\mathbf{Z}_i)}{\max(\mathbf{Z}_i) - \min(\mathbf{Z}_i)} \end{aligned} \quad (9)$$

Therefore, the i -th filter importance score can be denoted as:

$$\begin{aligned} \hat{\Omega}(\text{Conv}_i(I, :, :)) &= \text{DCTScore}(\text{Conv}_i(I, :, :)) \\ &= S_i \end{aligned} \quad (10)$$

This study observe that the importance score generated by a single filter with DCT is robust to the input images, it is very similar to HRank which is the ranks generated by a single filter. The visual statistical of the DCT importance score results are shown in Fig. 2. Hence, a small batch of input images can be used to accurately estimate the expectation of the feature map importance score. Then, define the importance score expectation of feature F_i as:

$$\begin{aligned} &\mathbb{E}_{I \sim P(I)}[\hat{\Omega}(\text{Conv}_{it}(I, :, :))] \\ &\approx \sum_{k=1}^g \text{DCTScore}(\text{Conv}_i(k, :, :)) \end{aligned} \quad (11)$$

Where g is the number of images to estimate the average importance score. Although different g will have different results, the difference is not obvious, and it is enough to

judge the importance. In the classification experiments, set $g = 128 \times 5$, while for salient detection, set $g = 16 \times 5$.

3.4 Pruning procedure

In a convolutional layer i , there are filters $\mathbf{W}_i$ and the feature maps F_i , prune $\mathbf{W}_i$ as follows: First, perform DCT in feature maps f_t^i to DCT map d_t^i in convolutional layer i in F_i . Then, caculate the importance score to form a importance score set $\mathbf{S}_i = \{s_1^i, s_2^i, \dots, s_t^i, \dots, s_{c_i}^i\} \in \mathbb{R}^{c_i}$ and re-rank the importance score set in decreasing order:

$$\hat{\mathbf{S}}_i = \{s_{I_1^i}^i, s_{I_2^i}^i, \dots, s_{I_t^i}^i, \dots, s_{I_{c_i}^i}^i\} \in \mathbb{R}^{c_i} \quad (12)$$

where I_t^i is the index of the t -th top value in $\mathbf{S}_i$. Third, determine the values of c_i^1 (number of preserved filters) and c_i^2 (number of pruned filters) to obtain the important filter set $\mathbf{W}_i^I = \{w_{I_1^i}^i, w_{I_2^i}^i, \dots, w_{I_{c_i^1}^i}^i\}$ where the importance score

of $w_{I_t^i}^i$ is $s_{I_t^i}^i$. It can obtain the filter set of unimportance score as the same way $\mathbf{W}_i^U = \{w_{I_{c_i^1+1}^i}^i, w_{I_{c_i^1+2}^i}^i, \dots, w_{I_{c_i^2}^i}^i\}$. Lastly,

remove the set $\mathbf{W}_i^U$ and fine-tune the network with $\mathbf{W}_i^I$ as the initialization. Figure 3 shows the visualization for DCT importance score caculation on feature maps from different convolutional layers with the input image of a “car”.

4 Experiments

In this Section 4.1 will firstly introduce the experimental setup. Then comparison the pruning results with the state-of-the-art classification task with variants pruning models and in salient object detection tasks with DCTPruning in Section 4.2. In Section 4.3, validate the effectiveness of DCTPruning firstly, then ablating the various importance score calculation methods.

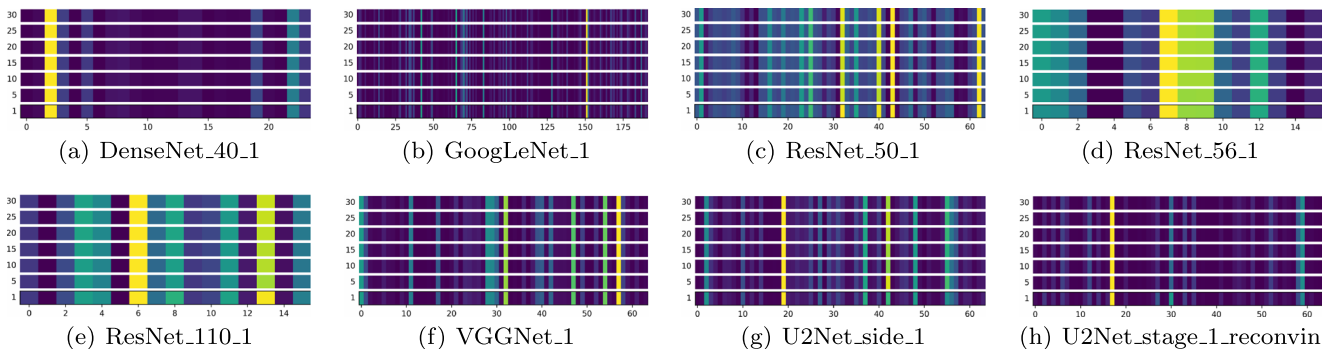
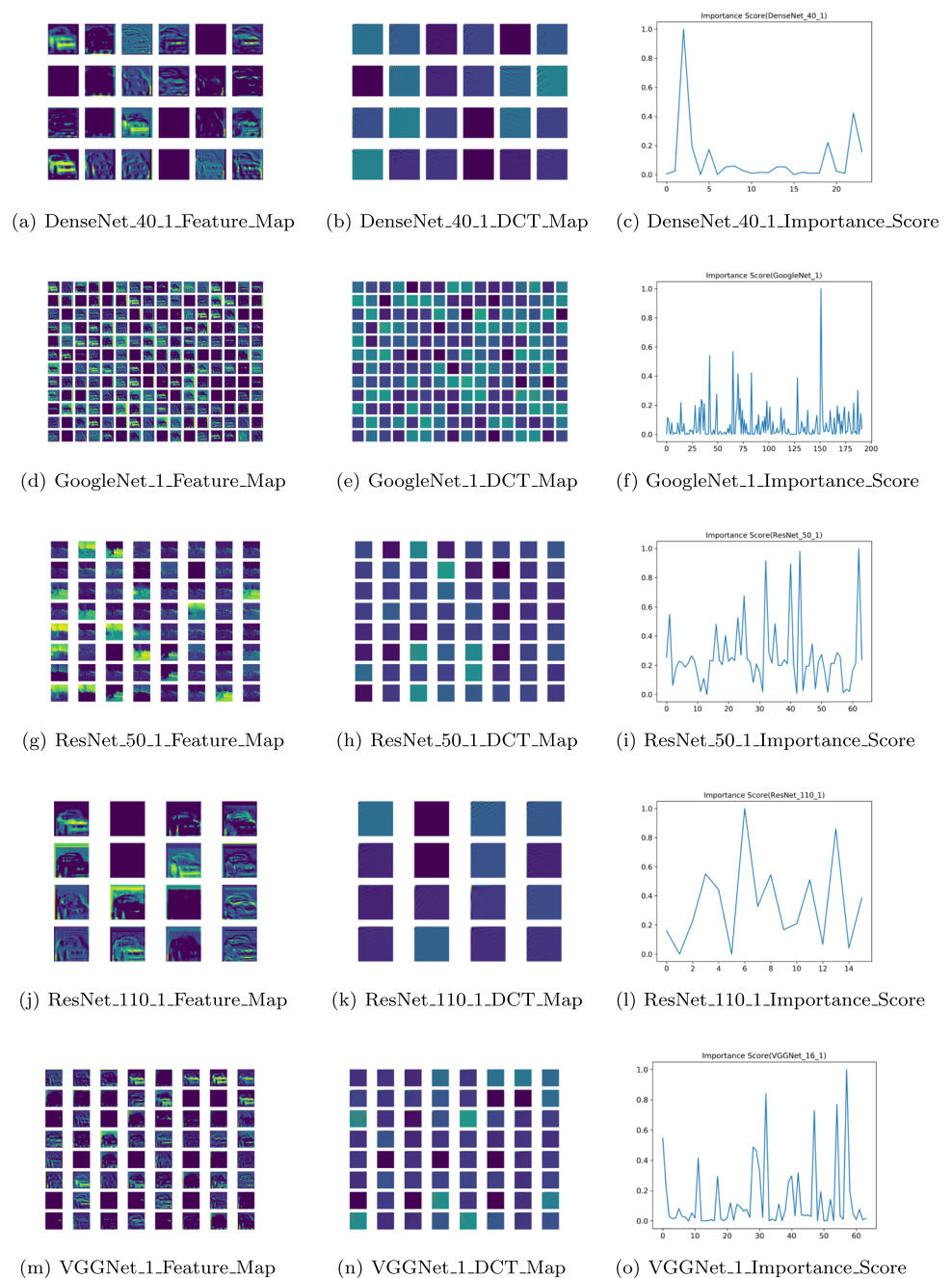


Fig. 2 Average DCT importance score of feature maps from different convolutional layers. DenseNet_40_1, GoogLeNet_1, ResNet_50_1, ResNet_56_1, ResNet_110_1 and VGGNet_1 are visualized on dataset CIFAR-10 for image classification task, y-axis denote the batches

with each batch size is 128 and x-axis denote the channels size on the feature map, while, U2Net_side_1 and U2Net_stage_1_reconvin are visualized on dataset DUTS for salient object detection task, each batch size is 16

Fig. 3 Visualization of DCT importance score calculation on feature maps for different convolutional layers. For each subfigure, the left is the feature map, the middle is the DCT map of the corresponding feature map, the right is the corresponding importance score



4.1 Experimental setup

For the classification task, adopt the widely-used protocols, i.e., number of parameters (denoted as Para) and required Float Points Operations (denoted as FLOPs), to evaluate the model size and computational requirement. To evaluate the task-specific capabilities, this paper provide top-1 accuracy (denoted as Top-1,%) of pruned models and the pruning rate (denoted as PR) on CIFAR-10 [36], and top-1 and top-5 (denoted as Top-5,%) accuracies of pruned models on ImageNet [39]. In particular, use the ratio (denoted as

Para/Top-1) of model size to Top1 accuracy to sort. We think that the ratio reflects the superiority of the model compression method to a certain extent.

From the experiments, get the experience that in different layers of the same network, the parameters of the front layers are more important than those of the back layers. Therefore, follow in the pruning experiment is that the pruning rate of the front layers is lower than that of the back layers. These numbers are usually determined according to the set pruning rate. Table 1 shows the detail of the pruning rate which used in the experiments. In addition, in order to

Table 1 Setting of pruning rate for models on different datasets

Model	FLOPs	Para,M	PruningRate [%]*Layers
Setting of Pruning Rate for Models on CIFAR-10			
DCTPruning-VGGNet	138.00M	4.61	[25]*7+[55]*5
DCTPruning-VGGNet	88.85M	2.00	[35]*7+[80]*5
DCTPruning-VGGNet	51.54M	1.09	[50]*7+[90]*5
DCTPruning-VGGNet	49.86M	0.87	[50]*7+[95]*5
DCTPruning-GoogLeNet	0.40B	2.10	[40]+[85]*2+[90]*5+[90]*2
DCTPruning-GoogLeNet	0.31B	1.84	[40]+[95]*2+[96]*7
DCTPruning-DenseNet-40	173.39M	0.62	[0]*1+[20]*12+[0]*1+[20]*12+[0]*1+[20]*12
DCTPruning-DenseNet-40	113.08M	0.40	[0]*1+[40]*12+[0]*1+[40]*12+[0]*1+[40]*12
DCTPruning-ResNet-56	90.35M	0.66	[0]*1+[40]*2+[50]*9+[60]*9+[70]*9
DCTPruning-ResNet-56	34.79M	0.24	[0]*1+[18]*29
DCTPruning-ResNet-110	135.88M	1.00	[0]*1+[20]*2+[30]*18+[40]*18+[39]*19
DCTPruning-ResNet-110	64.66M	0.47	[0]*1+[40]*2+[50]*18+[70]*36
Setting of Pruning Rate for Models on ImageNet			
DCTPruning-ResNet-50	2.41M	15.79	[0]*1+[20]*3+[25]*7+[30]*9
DCTPruning-ResNet-50	7.45M	1.06	[0]*1+[10]*3+[40]*7+[40]*9
DCTPruning-VGGNet-D	7.08M	128.78	[35]*1+[25]*1+[35]*1+[25]*1+[45]*2+[35]*1+[45]*2+[35]*1+[50]*2

facilitate the comparison, usually set the maximum pruning rate slightly higher than other methods to observe whether the proposed method can recover to a better accuracy. Some specific layers need to be kept or cannot be pruned. For example, in ResNet, the input and output of the shortcut part need to be consistent, so the upper layer and the lower layer need to maintain the same pruning rate. It also keep full connection layers not to be pruned since the importance calculation is not effective in full connection layers.

For salient object detection, in terms of performance measurement, use BASNet's method [37] to measure average Mean Absolute Error (aveMAE) on DUTS [40] after pruning, Max F-measure (maxF), and relaxed F-measure of the boundary (relaxF). The smaller aveMAE, the better the performance of the model; the larger the maxF and relaxF, the better the performance of the model.

4.2 Comparison with state-of-the-art

4.2.1 Pruning models for classification on CIFAR-10

This study compare the performance of different pruning methods in several popular convolutional neural networks (CNNs) on CIFAR-10 dataset. The CNNs include VGG-16 [28], GoogLeNet [29], DenseNet-40 [30], ResNet-56 [31] and ResNet-110 [31]. Following HRank [5] configuration, changed the output of the original GoogLeNet to fit the class number in CIFAR-10 and 40 layers with a growth rate of 12 for DenseNet-40.

VGGNet Table 2 shows the comparison results of pruning VGGNet on CIFAR-10. Using VGG-16 for the testing. Compared with the original VGG-16, DCTPruning provides the FLOPs (56.0% reduced) and parameters (69.2% reduced) are reduced by more than half, and the Top-1 accuracy (93.97% vs 93.96%) is even higher than the original model after fine-tuning. Although the parameters are reduced by 94.2% and FLOPs are reduced by 84.1%, the accuracy is only reduced by 1.15% (92.81% vs 93.96%). Compared with HRank [5], DCTPruning yields a better compression and acceleration rate while maintaining a higher accuracy (For example, 84.1% vs. 76.5% in FLOPs reduction, 94.2% vs. 88.1% in parameters reduction, and 92.53% vs. 91.23% in top-1 accuracy compared with HRank-1.78M). Compared with other excellent methods [1–4], DCTPruning also has better compression and acceleration, while maintaining a higher accuracy (93.43% vs. 93.43% by HRank-2.51M, 92.34% by HRank-2.64M, 90.73% by GAL-0.1, 92.03% by GAL-0.05, 93.18% by Zhao et al, 93.02% by SSS and 93.40% by L1).

GoogLeNet Table 3 shows the comparison results of pruning GoogLeNet on CIFAR-10. Compared with original GoogLeNet, DCTPruning provides the FLOPs are reduced by 74.1% reduced and parameters are reduced by 66.0%, and the Top-1 accuracy is only reduced by 0.38% (94.67% vs 95.05%). Compared with HRank [5], DCTPruning has better performance to be effectively applied to compressing neural networks with inception modules (79.6% vs. 70.4%

Table 2 Pruning results of VGGNet on the dataset of CIFAR-10

Model	Para/Top-1	Top-1, %	FLOPs(PR, %)	Para, M(PR, %)
Original VGGNet	0.063	93.96	313.73M(0.0)	14.98(0.0)
L1 [1]	0.173	93.40	206.00M(34.3)	5.40(64.9)
DCTPruning(Ours)	0.204	93.97	138.00M(56.0)	4.61(69.2)
SSS [2]	0.237	93.02	183.13M(41.6)	3.93(73.8)
Zhao et al [3]	0.238	93.18	190.00M(39.1)	3.92(73.8)
GAL-0.05 [4]	0.274	92.03	189.49M(39.6)	3.36(77.6)
GAL-0.1 [4]	0.340	90.73	171.89M(45.2)	2.67(82.2)
HRank-2.64M [5]	0.350	92.34	108.61M(65.3)	2.64(82.4)
HRank-2.51M [5]	0.372	93.43	145.61M(53.5)	2.51(82.9)
DCTPruning(Ours)	0.467	93.43	88.85M(71.7)	2.00(86.6)
HRank-1.78M [5]	0.513	91.23	73.70M(76.5)	1.78(88.1)
DCTPruning(Ours)	0.851	92.78	51.54M(83.6)	1.09(92.7)
DCTPruning(Ours)	1.067	92.81	49.86M(84.1)	0.87(94.2)

in FLOPs reduction, 70.1% vs. 69.8% in parameters reduction, and 94.27% vs. 94.07% in top-1 accuracy compared with HRank-1.86M). Compared with L1 [1], GAL-0.05 [4], GAL-ApoZ [6] and the Random methods, DCTPruning shows a better compression and acceleration rate while maintaining a higher accuracy (94.62% vs. 94.54% by L1, 93.93% by GAL-0.05, 92.11% by GAL-ApoZ, 94.54% by Random).

DenseNet-40 Table 4 shows the comparison results of pruning GoogLeNet on CIFAR-10. Compared with original DenseNet-40, DCTPruning provides the FLOPs are reduced by 38.5% reduced and parameters are reduced by 40.4%, and the Top-1 accuracy is only reduced by 0.49% (94.32% vs 94.81%). Compared with HRank [5], GAL [4], Liu et-40% [7], HRank [5] and Zhao et al.[3], DCT-Pruning keeps a better accuracy while reducing a higher parameters compression (62.6% vs. 35.6% by GAL-0.01, 36.5% by HRank-0.66M, 36.5% by Liu et-40, 53.8% by HRank-0.48M, 56.7% by GAL-0.05 and 59.6% by Zhao

et-al). It shows DCTPruning can effectively be applied to DenseNet.

ResNet Table 5 shows the comparison results of pruning ResNet on CIFAR-10. First analyze the results of ResNet-56, and then analyze the results of ResNet-110. Compared with the original ResNet-56, DCTPruning provides the FLOPs are reduced by 28.0% reduced and parameters are reduced by 22.4%, but the Top-1 accuracy is increased by 0.7% (93.96% vs 93.26%). Compared with GAL [4], L1 [1] HRank [5] and NISP [8], DCTPruning keeps a better accuracy while reducing a higher parameters compression (71.8% vs. 11.8% by GAL-0.6, 14.1% by L1, 16.8% by HRank-0.73M, 42.4% by NISP, 42.4% by HRank-0.49M and 65.9% by GAL-0.8 and 68.1% by HRank-0.27M). For ResNet-110, DCTPruning also provides the top-1 accuracy is increased by 0.8% (94.26% vs 93.50%) while FLOPs are reduced by 46.3% and parameters are reduced by 41.9%. And it also achieved the best ratio of parameter size to Top-1 accuracy which keeps a better accuracy for ResNet-110

Table 3 Pruning results of GoogLeNet on the dataset of CIFAR-10

Model	Para/Top-1	Top-1, %	FLOPs(PR, %)	Para, M(PR, %)
Original GoogLeNet	0.155	95.05	1.52B(0.0)	6.15(0.0)
Random	0.264	94.54	0.96B(36.8)	3.58(41.8)
L1 [1]	0.269	94.54	1.02B(32.9)	3.51(42.9)
GAL-0.05 [4]	0.301	93.93	0.94B(38.2)	3.12(49.3)
GAL-ApoZ [6]	0.323	92.11	0.76B(50.0)	2.85(53.7)
HRank-2.74M [5]	0.345	94.53	0.69B(54.9)	2.74(55.4)
DCTPruning(Ours)	0.451	94.67	0.40B(74.1)	2.10(66.0)
HRank-1.86M [5]	0.506	94.07	0.45B(70.4)	1.86(69.8)
DCTPruning(Ours)	0.512	94.27	0.31B(79.6)	1.84(70.1)

Table 4 Pruning results of DenseNet-40 on the dataset of CIFAR-10

Model	Para/Top-1	Top-1,%	FLOPs(PR,%)	Para,M(PR,%)
Original DenseNet-40	0.912	94.81	282.00M(0.0)	1.04(0.0)
GAL-0.01 [4]	1.407	94.29	182.92M(35.1)	0.67(35.6)
HRank-0.66M [5]	1.428	94.24	167.41M(40.6)	0.66(36.5)
Liu et al.-40% [7]	1.437	94.81	190.00M(32.6)	0.66(36.5)
DCTPruning(Ours)	1.521	94.32	173.39M(38.5)	0.62(40.4)
HRank-0.48M [5]	1.952	93.68	110.15M(61.0)	0.48(53.8)
GAL-0.05 [4]	2.078	93.53	128.11M(54.7)	0.45(56.7)
Zhao et al. [3]	2.218	93.16	156.00M(44.8)	0.42(59.6)
DCTPruning(Ours)	2.339	93.54	113.08M(60.7)	0.40(62.6)

pruning. It shows DCTPruning can effectively be applied to ResNet.

4.2.2 Pruning models for classification on ImageNet

This part first compare the performance of different pruning methods in ResNet-50 on ImageNet dataset. The comparison results are shown in Table 6. Compared with the original ResNet-50, DCTPruning provides the FLOPs are reduced by 74.1% reduced and parameters are reduced by 70.8%, and the Top-1 accuracy is only reduced by 3.84% (72.31% vs 76.15%), the Top-5 accuracy is only reduced by 2.3% (90.77% vs 92.87%). Compared with HRank [5], ThiNet-50

[10], GAL [4], SSS [2], GDP [11], DCTPruning keeps the best ratio coefficient of parameters vs Top-1 accuracy. (0.103 vs 0.084 by HRank-8.27M, 0.079 by ThiNet-50, 0.068 by GAL-1-joint, 0.052 by HRank-13.77M, 0.048 by GAL-1, 0.046 by HRank-16.15M, 0.046 by SSS-26, 0.040 by SSS-32, 0.037 by GAL-0.5-joint, 0.034 by GAL-0.5).

Then further compare the performance of different pruning methods in VGGNet-D [28] on ImageNet dataset. The special feature of this network is that the full connection layer occupies the vast majority of parameters. For a fair comparison, do the same as other methods without prune the full connection layers. Thus, the reduction of parameters is limited. But still keep the high accuracy of Top-1 and

Table 5 Pruning results of ResNet on the dataset of CIFAR-10

Model	Para/Top-1	Top-1,%	FLOPs(PR,%)	Para,M(PR,%)
Original ResNet-56	1.097	93.26	125.49M(0.0)	0.85(0.0)
He et al. [9]	-	90.80	62.00M(50.6)	-
GAL-0.6 [4]	1.240	92.98	78.30M(37.6)	0.75(11.8)
L1 [1]	1.275	93.06	90.90M(27.6)	0.73(14.1)
HRank-0.73M [5]	1.317	93.52	88.72M(29.3)	0.71(16.8)
DCTPruning(Ours)	1.424	93.96	90.35M(28.0)	0.66(22.4)
NISP [8]	1.898	93.01	81.00M(35.5)	0.49(42.4)
HRank-0.49M [5]	1.901	93.17	62.72M(50.0)	0.49(42.4)
GAL-0.8 [4]	3.116	90.36	49.99M(60.2)	0.29(65.9)
HRank-0.27M [5]	3.360	90.72	32.52M(74.1)	0.27(68.1)
DCTPruning(Ours)	3.842	92.21	34.79M(72.5)	0.24(71.8)
Original ResNet-110	0.544	93.50	252.89M(0.0)	1.72(0.0)
L1 [1]	0.804	93.30	155.00M(38.7)	1.16(32.6)
HRank-1.04M [5]	0.906	94.23	148.70M(41.2)	1.04(39.4)
DCTPruning(Ours)	0.943	94.26	135.88M(46.3)	1.00(41.9)
GAL-0.5 [4]	0.974	92.55	130.20M(48.5)	0.95(44.8)
HRank-0.70M [5]	1.334	93.36	105.70M(58.2)	0.70(59.2)
HRank-0.53M [5]	1.748	92.65	79.30M(68.6)	0.53(68.7)
DCTPruning(Ours)	1.984	93.27	64.66M(74.4)	0.47(72.7)

Table 6 Pruning results of ResNet50 and VGGNet on the dataset of ImageNet

Model	Para/Top-1	Top-1,%	Top-5,%	FLOPs,B(PR,%)	Para,M(PR,%)
ResNet-50 [10]	0.030	76.15	92.87	4.09(0.0)	25.50(0.0)
He et al. [9]	-	72.30	90.80	2.73(33.3)	-
GDP-0.6 [11]	-	71.19	90.71	1.88(54.0)	-
GDP-0.5 [11]	-	69.58	90.14	1.57(61.6)	-
GAL-0.5 [4]	0.034	71.95	90.94	2.33(43.0)	21.20(16.9)
GAL-0.5-joint [4]	0.037	71.80	90.82	1.84(55.0)	19.31(24.3)
SSS-32 [2]	0.040	74.18	91.91	2.82(31.1)	18.60(27.1)
SSS-26 [2]	0.046	71.82	90.79	2.33(43.0)	15.60(38.8)
HRank-16.15M [5]	0.046	74.98	92.33	2.30(43.8)	16.15(36.7)
GAL-1 [4]	0.048	69.88	89.75	1.58(61.4)	14.67(42.5)
DCTPruning(Ours)	0.048	75.12	92.15	2.41(41.1)	15.79(38.1)
HRank-13.77M [5]	0.052	71.98	91.01	1.55(62.1)	13.77(46.0)
GAL-1-joint [4]	0.068	69.31	89.12	1.11(72.9)	10.21(60.0)
ThiNet-50 [10]	0.079	68.42	88.30	1.10(73.1)	8.66(66.0)
HRank-8.27M [5]	0.084	69.10	89.58	0.98(76.0)	8.27(67.6)
DCTPruning(Ours)	0.103	72.32	90.77	1.06(74.1)	7.45(70.8)
Original VGGNet-D [28]	2.02	68.34	88.44	15.18(0.0)	138.36(0.00)
SSS [2]	-	68.53	88.20	7.67(49.5)	-
Molchanov et al. [34]	-	-	84.50	~8(47.3)	-
He et al. [9]	-	67.80	88.10	7.03(53.7)	-
DCTPruning(Ours)	1.88	68.50	88.72	7.08(53.4)	128.78(0.08)

Top-5 in the case of large FLOPs reduction (53.4%). It shows DCTPruning has the capability that can be applied to large-scale image datasets.

4.2.3 Pruning U2NETP for salient object detection on DUTS

Saliency detection is different from the task of image classification. Its main purpose is to detect the most significant region in the image by simulating human visual characteristics. The shape of this region may be various, so the output of the model is usually the mask of the corresponding size, and the value of the mask represents whether the corresponding position in the original image is significant. Similar to saliency detection, the “Image-In-Image-Out” also includes semantic segmentation, instance segmentation, image restoration, and so on. However, previous papers on pruning have rarely been verified on such tasks. Use DCT pruning to verify the saliency object detection and prove that the proposed method is still suitable for the “Image-In-Image-Out” task.

This experiment test the efficiency model U2NETP proposed in the latest saliency detection framework U2NET [38]. Its model size is already very small among many of the other state-of-the-art saliency detection models. For salient object detection, in terms of performance measurement, use BASNet’s method [37] to measure average Mean Absolute

Error (aveMAE) on DUTS [40] after pruning, Max F-measure (maxF), and relaxed F-measure of the boundary (relaxF). The smaller aveMAE, the better the performance of the model; the larger the maxF and relaxF, the better the performance of the model. Table 7 shows the comparison of U2NETP pruned by DCT with the original model and many other state-of-the-art saliency detection models. This study found that compared with the original model, reduced the model by 59.6% (1.9 vs 4.7) by pruning, but still maintained a high performance, and even relaxF achieved the highest among the models compared. Compared with other models, U2NETP after pruning has great advantages in model size (1.9 vs 112.1 by MDF [41], 117.9 by UCF [42], 132.6 by Amulet [43], 428.0 by NLDF [44], 81.0 by RAS [45], 48.6 by MSW [46], 215.0 by R3Net [47], 189.0 by SRM [48], 348.5 by BASNet [37]), but other performance still has advantages. This proves that the proposed pruning method is still effective for tasks like “image in and image out” tasks.

4.3 Ablation study

4.3.1 Effectiveness ablation

In the process of pruning, use DCT to calculate the importance of the channel to select the parameters of the important channel to initialize the new network. However, how to use

Table 7 Pruning results of U2NETP on the dataset of DUTS

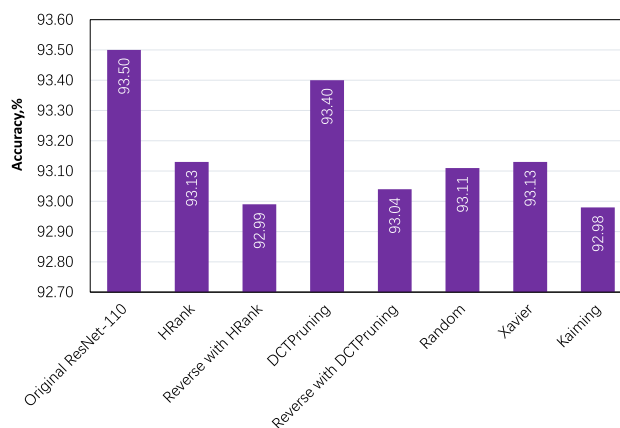
Model	Backbone	Size(MB), ↓	aveMAE, ↓	maxF, ↑	relaxF, ↑
MDF [41]	AlexNet	112.1	0.099	0.729	0.447
UCF [42]	VGG-16	117.9	0.112	0.773	0.518
Amulet [43]	VGG-16	132.6	0.084	0.778	0.568
NLDF [44]	VGG-16	428.0	0.065	0.813	0.591
RAS [45]	VGG-16	81.0	0.059	0.831	0.656
MSW [46]	Dense-169	48.6	0.908	0.767	0.376
R3Net [47]	ResNeXt	215.0	0.058	0.828	0.601
SRM [48]	ResNet-50	189.0	0.058	0.826	0.592
BASNet [37]	ResNet-34	348.5	0.047	0.860	0.758
Original U2NETP [38]	RSU	4.7	0.054	0.852	0.723
HRankPruningU2NETP [5]	RSU	1.9	0.071	0.811	0.767
DCTPruningU2NETP(Ours)	RSU	1.9	0.069	0.815	0.776

experiments to prove that the parameters selected by this method are effective? This study design experiments to verify the effectiveness of DCTPruning on both classification tasks and saliency object detection tasks.

Effectiveness on classification task Design an experiment for Pruning ResNet-110 on CIFAR-10, without changing any hyperparameters, used different initialization methods to observe the top-1 accuracy on CIFAR-10. The original ResNet-110's parameter size is 1.72MB, and the FLOPs are 252.89M. After pruning, the size of the parameter is 0.54M (pruning rate is 68.6%), and the FLOPs is 71.60M (pruning rate is 71.69%). Figure 4 shows the effectiveness test with various initialization methods for the pruned ResNet-110 on CIFAR-10. These methods include: 1. *HRank*, initializing the pruned ResNet-110 according to the rank value after calculating the feature's rank according to the HRank [5]; 2. *Reverse with HRank*, using HRank to calculate the rank of the channel, and then sort and initialize the pruned ResNet-110 according to a reversed rank value; 3. *DCTPruning*, initializing the pruned ResNet-110 according to the importance after calculating the importance according to the DCT; 4. *Reverse with DCTPruning*, using DCT to calculate the importance of the channel, and then sort and initialize the pruned ResNet-110 according to a reversed important degree; 5. *Random*, randomly selecting the parameters in the pre-training model to initialize the pruned ResNet-110; 6. *Xavier*, does not select parameters from the pre-training model, but initializes by xavier method [49]. 7. *Kaiming*, does not select parameters from the pre-training model but initializes by kaiming method [50]. The results show DCTPruning is the most effective method compared with others. Although HRank is also effective, this effect is not obvious compared with DCTPruning: the pruned accuracy is 0.27% higher

than that of HRank under the same conditions (93.40% vs 93.13%); DCTPruning is higher than Reverse with DCTPruning 0.36% (93.40% vs 93.04%), while, HRank is only higher than the Reverse with HRank 0.14% (93.13% vs 92.99%).

Effectiveness on saliency object detection task This study designed such an experiment on U2NETP pruning: without changing any hyperparameters, using different initialization methods to observe the training process of pruned U2NETP. These methods include: *Random*, *Reverse*, *DCTPruning*, *Kaiming*, *Xavier* and *HRank*. These methods' definitions are the same as in effectiveness on classification task. Figure 5 shows the training process of initializing the pruned U2NETP by these methods, the horizontal axis represents the number of iterations of training, and the vertical axis represents the loss. The results show that the model converges faster, and the final loss is the lowest after the parameters of the pruned U2NETP are

**Fig. 4** Effectiveness test with various initialization methods for the pruned ResNet-110 on CIFAR-10

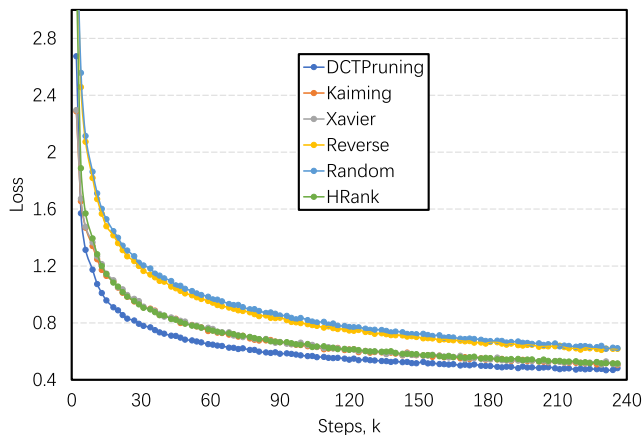


Fig. 5 Effectiveness test with various initialization methods for the pruned U2NETP on DUTS

initialized by DCTPruning. At the same time, it also finds an interesting phenomenon that *Kaiming* and *Xavier* are better than *Random* and *Reverse* initialization. What can be explained here is that if the unimportant parameters are selected to initialize the network in the original pre-training model, the pruned U2NETP may be misguided to train, resulting in a longer convergence time. This also proves that the proposed importance calculation is effective.

4.3.2 Various importance score calculation methods ablation

The proposed method of channel importance is to sum the squares of all the points after DCT transformation on the feature map and then add them. In this paper, use (5) to calculate it, name this method as “squ.add”. The function of the square is to widen the gap between “important” and “unimportant”. Of course, it can also have many other methods to calculate while ensuring that other processes remain unchanged. By changing (5), derive other calculation methods: “abs.add” is used to calculate the absolute values, and then add them; “squ.add” is used to calculate the squared values, and then add them (the method in this paper); “th_abs_cnt” is used to calculate the absolute values, and then count the number of values higher than a certain threshold (set 0.001 in the experiment), “th_squ_cnt” is used to calculate the squared values, and then count the number of values higher than a certain threshold (set 0.001 in the experiment). Calculate the importance of the first layer in the VGG-16 network according to these methods and normalize these values to get Fig. 6. The importance scores obtained by various methods are not the same, but their trend is always the same, which shows that the calculation of these importance scores is effective because it only needs to sort the channels and do not need to know

the absolute value of its importance. Table 8 shows the configurations of pruning rates for VGG-16 on CIFAR-10 dataset while Fig. 7 shows the corresponding pruning results. The results show that the accuracy of different importance score calculation methods under the same training parameters is different under the same pruning rate, but the difference is not significant. However, the average accuracy of the “squ.add” method is better than other methods.

4.3.3 Applicability discussion

In some convolution networks, convolution modules usually include convolution operations, normalization, and activation functions. For example, the convolution module of classification network is composed of “conv + BN + relu”, where “conv” denotes “convolution operation”, “BN” denotes “batch normalization” and “relu” denotes “ReLU” activation function. In the experiments, in the convolutional structure of “conv + BN + relu”, found that use HRank [5] effective only after “relu”; if move the calculation before “relu” or “BN”, all channels of the feature map are full rank, lead to can not use rank to measure the importance of channels. That means HRank [5] might be limited to networks equipped with “relu” layers. However, the proposed method has no such limitation and can be applied to any form of convolutional network. Moreover, for channel pruning, the calculation of importance should be consistent in the structure, proved that the proposed method is more suitable for channel importance measurement compared with HRank. Figure 8 shows the results of importance score calculation in different positions of the convolutional structure with DCTPruning and HRank. The channel importance calculated by DCTPruning before or after “relu” is consistent. While, the channel importance calculated by HRank is seriously inconsistent: before “relu”, all channels of the feature map are full rank make it can not distinguish the importance channels. The proposed method has no such limitation and can be applied to any convolutional network. Moreover, for channel pruning, the calculation of importance should be consistent in the structure, proving that the proposed method is more suitable for channel importance measurement than HRank.

4.4 Practical speedup

The practical speedup depends on intermediate layers bottleneck, availability of data (batch size), and the number of CPU/GPU cores available [55]. Select the pruned VGGNet with 49.86M FLOPs (pruning rate is 84.1%) as the DCTPruning-VGGNet in Fig. 9. To achieve a fair comparison, set up a 512K (In this way, the image data set can be divided by a batch size of 64, 128, and 512) image

Fig. 6 Various importance score calculation methods for the first layer of VGG-16.

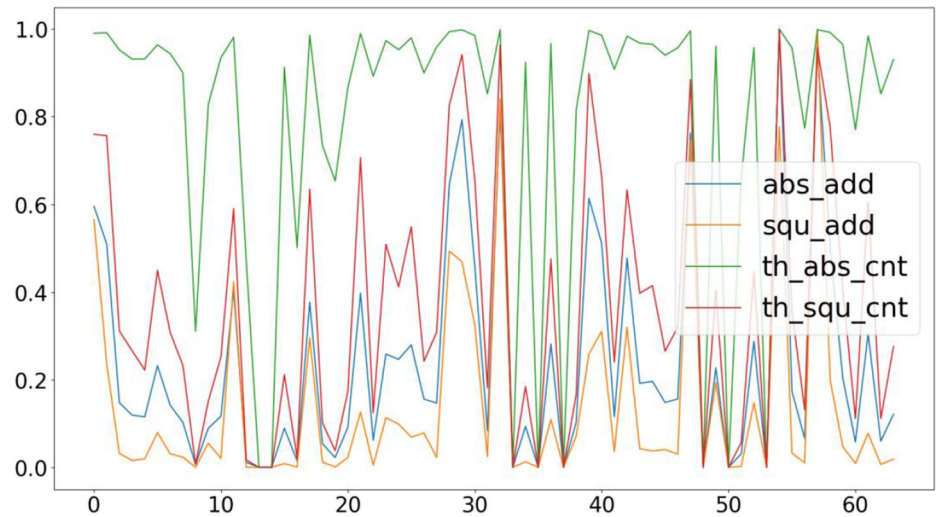


Table 8 Configurations of pruning VGGNet on CIFAR-10

Configurations	FLOPs,M	ModeSize,M	PruningRate [%]*Front Layers+[%]*Back Layers
Configure1	138.00	4.61	[25]*7+[55]*5
Configure2	117.24	3.89	[30]*7+[60]*5
Configure3	99.91	3.25	[35]*7+[65]*5
Configure4	83.34	2.65	[40]*7+[70]*5
Configure5	68.95	2.13	[45]*7+[75]*5
Configure6	56.24	1.67	[50]*7+[80]*5
Configure7	43.53	1.26	[55]*7+[85]*5
Configure8	33.56	0.92	[60]*7+[90]*5

Fig. 7 Pruning results of VGGNet on CIFAR-10 with variants importance score caculation methods

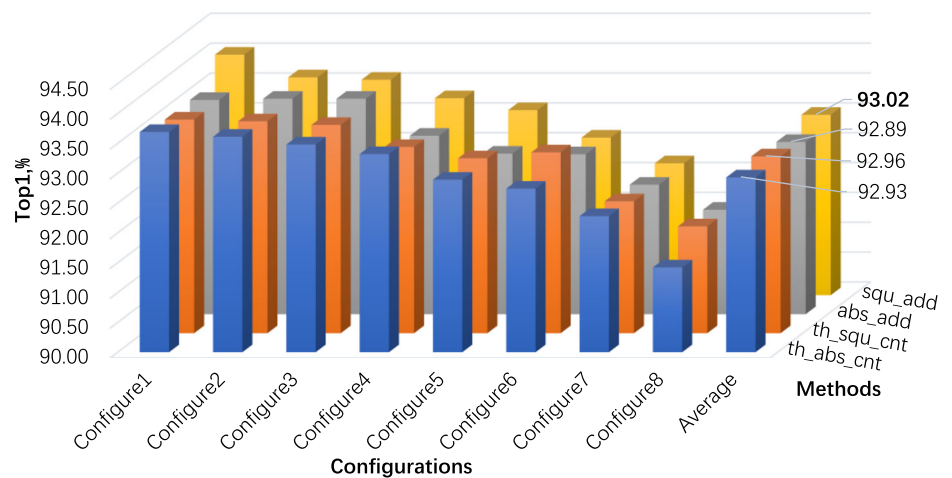
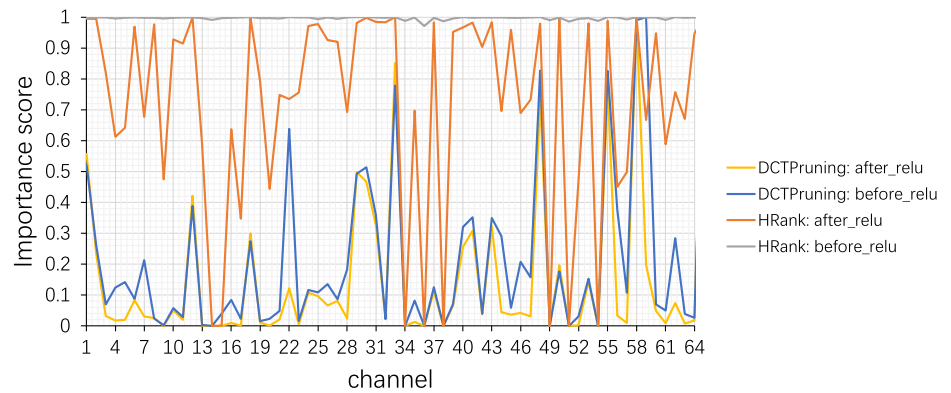


Fig. 8 The importance score calculation in different positions of the convolutional structure with DCTPruning and HRank



data set, and the size of each image is 32×32 for testing. Figure 9 shows the practical speedup on CPU and GPU with different batch size. Although it reduces the FLOPs by about 84.1%, the best speedup in practice is only about 3.7 times due to various conditions.

4.5 Fine-tuning vibrations

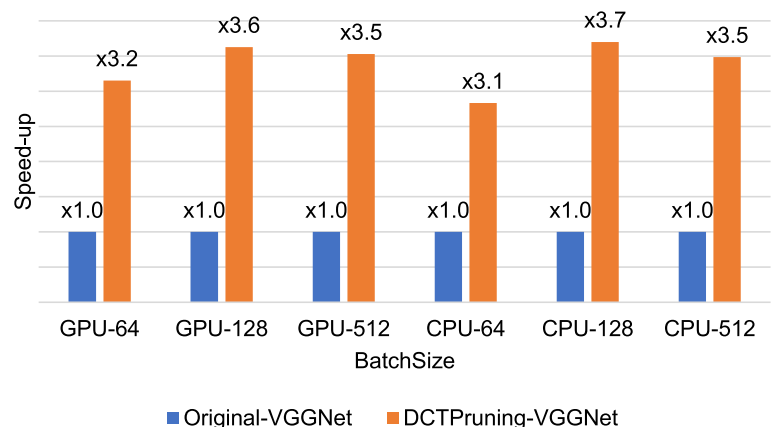
In the pruning process, the last step is fine-tuning. Then verify the tuned accuracy on the corresponding dataset. The experiment fixed the number of iterations (for example, fixed batch size as 256 and epochs as 150 when fine-tuning on cifar10) and selected the top-1 best accuracy as the final comparison accuracy. However, the top-1 accuracy may be different in different optimization processes, which is caused by the optimization process of training. In order to verify this, selected the pruned DCTPruning-VGGNet(with 49.86M FLOPs , pruning rate is 84.1%) to fine-tuning 10 times and obtained the following top-1 accuracy: [92.65%, 92.63%, 92.41%, 92.85%, 92.47%,

92.98%, 92.57%, 92.57%, 92.59%, and 92.66%]. The average top-1 accuracy is 92.61%, and the standard deviation is 0.168. Even if there are some deviations, on the whole, satisfactory results are obtained for the DCTPruning.

5 Conclusions

This paper presents a novel filter pruning method, DCTPruning, based on discrete cosine transform on feature maps. DCTPruning utilizes the discrete cosine transform on feature maps with original pre-training models to calculate the filters' importance score, then decide which filters should be pruned firstly. The experimental results show that DCTPruning with various models on classification tasks effectively reduces the computational complexity and model size. Additionally, the proposed method also effectively reduces the computational complexity and model size on "Image-In-Image-Out" tasks with the experiment on salient object detection.

Fig. 9 Speedup comparison of original VGGNet and DCTPruning VGGNet on Intel(R) Xeon(R) Gold 6240 CPU @ 2.60GHz and NVIDIA TITAN RTX GPU with different batch size



Acknowledgements This work was supported in part by the National Natural Science Foundation of China under Grant No. 61772352; National Key Research and Development Project under Grant No. 2020YFB1711800 and 2020YFB1707900; the Science and Technology Project of Sichuan Province under Grant No. 2019YFG0400, 2021YFG0152, 2020YFG0479, 2020YFG0322, 2020GFW035, and the R&D Project of Chengdu City under Grant No. 2019-YF05-01790-GX.

References

- Li H, Kadav A, Durdanovic I, Samet H, Graf HP (2017) Pruning filters for efficient convnets. In: Proceedings of ICLR
- Huang Z, Wang N (2018) Data-driven sparse structure selection for deep neural networks. In: Proceedings of ECCV, pp 304–320
- Zhao C, Ni B, Zhang J, Zhao Q, Zhang W, Tian Q (2019) Variational convolutional neural network pruning. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 2780–2789
- Lin S, Ji R, Yan C, Zhang B, Cao L, Ye Q, Huang F, Doermann D (2019) Towards optimal structured cnn pruning via generative adversarial learning. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 2780–2789
- Lin M, Ji R, Wang Y et al (2020) HRank: Filter Pruning using High-Rank Feature Map. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 1529–1538
- Hu H, Peng R, Tai Y, Tang C (2016) Network trimming: A data-driven neuron pruning approach towards efficient deep architectures, arXiv:1607.03250
- Liu Z, Li J, Shen Z, Huang G, Yan S, Zhang C (2017) Learning efficient convolutional networks through network slimming. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 2736–2744
- Yu R, Li A, Chen CF, Lai JH, Morariu VI, Han X, Gao M, Lin CY, Davis LS (2018) Nisp: Pruning networks using neuron importance score propagation. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 9194–9203
- He Y, Zhang X, Sun J (2017) Channel pruning for accelerating very deep neural networks. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 1389–1397
- Luo JH, Wu J, Lin W (2017) Thinet: A filter level pruning method for deep neural network compression. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 5058–5066
- Lin S, Ji R, Li Y, Wu Y, Huang F, Zhang B (2018) Accelerating convolutional networks via global & dynamic filter pruning. In: International Joint Conference on Artificial Intelligence (IJCAI), pp 7
- Denil M, Shakibi B, Dinh L, de Freitas N et al (2013) Predicting parameters in deep learning, arXiv:1306.0543
- LeCun Y, Denker JS, Solla SA, Howard RE, Jackel LD (1990) Optimal brain damage. In: Advances in neural information processing systems, pp 598–605
- Hassibi B, Stork DG (1993) Second order derivatives for network pruning: Optimal brain surgeon. Morgan Kaufmann, pp 164–171
- Han S, Pool J, Tran J, Dally W (2015) Learning both weights and connections for efficient neural network. In: Proceedings of NIPS, pp 1135–1143
- Wei W et al (2016) Learning Structured Sparsity in Deep Neural Networks. In: Proceedings of NIPS, pp 2074–2082
- Ahmed N, Natarajan T, Rao KR (1974) Discrete cosine transform. IEEE Trans Comput:90–93
- Wallace GK (1991) The jpeg still picture compression standard. Commun ACM 34(4):30–44
- Rabbani M, Joshi R (2002) An overview of the jpeg 2000 still image compression standard. Signal Process: Image Commun 17(1):3–48
- Chen W et al (2016) Compressing convolutional neural networks in the frequency domain, the 22nd ACM SIGKDD International Conference ACM
- Chen W, Wilson JT, Tyree S, Weinberger KQ, Chen Y (2015) Compressing neural networks with the hashing trick. In: Proceedings of ICML, pp 2285–2294
- Han S, Mao H, Dally WJ (2016) Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding. In: Proceedings of ICLR
- Courbariaux M, Bengio Y (2016) Binarynet: Training deep neural networks with weights and activations constrained to +1 or -1, arXiv:1602.02830
- Baker B, Gupta O, Naik N et al (2016) Designing neural network architectures using reinforcement learning. [J]. arXiv:1611.02167
- Real E, Moore S, Selle A et al (2017) Large-scale evolution of image classifiers. [J]. arXiv:1703.01041
- Hinton G, Vinyals O, Dean J (2015) Distilling the knowledge in a neural network. Computer ence 14.7:38–39
- Huang Z, Wang N (2017) Like what you like: Knowledge distill via neuron selectivity transfer. [J]. arXiv:1707.01219
- Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. [J]. arXiv:1409.1556
- Szegedy C, Liu W, Jia Y et al (2015) Going deeper with convolutions. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit (CVPR), pp 1–9
- Huang G, Liu Z, Laurens VDM et al (2017) Densely Connected Convolutional Networks. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 4700–4708
- He K, Zhang X, Ren S et al (2016) Deep Residual Learning for Image Recognition. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 770–778
- Liu B, Wang M, Foroosh H et al (2015) Sparse Convolutional Neural Networks. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 806–814
- Lee N, Ajanthan T, Torr PHS (2018) Snip: Single-shot network pruning based on connection sensitivity. [J]. arXiv:1810.02340
- Molchanov P, Tyree S, Karras T et al (2016) Pruning convolutional neural networks for resource efficient inference. [J]. arXiv:1611.06440
- Ye J, Lu X, Lin Z et al (2018) Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers. [J]. arXiv:1802.00124
- Krizhevsky A, Hinton G et al (2009) Learning multiple layers of features from tiny images, Technical report, Citeseer, pp 2, 5
- Qin X, Zhang Z, Huang C et al (2019) BASNet: Boundary-aware Salient Object Detection. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit (CVPR), pp 7479–7489
- Qin X, Zhang Z, Huang C et al (2020) U2-net: Going deeper with nested U-structure for salient object detection. [J] Pattern Recogn 106:107404
- Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M et al (2015) Imagenet large scale visual recognition challenge. International J Comput Vis (IJCV) 2:5
- Wang L, Lu H, Wang Y, Feng M, Wang D, Yin B, Ruan X (2017) Learning to detect salient objects with image-level supervision. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 136–145
- Li G, Yu Y (2016) Visual saliency detection based on multiscale deep cnn features. IEEE Trans Image Process 25(11):5012–5024

42. Zhang P, Wang D, Lu H, Wang H, Yin B (2017) Learning uncertain convolutional features for accurate saliency detection. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 212–221
43. Zhang P, Wang D, Lu H, Wang H, Ruan X (2017) Amulet: Aggregating multi-level convolutional features for salient object detection. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 202–211
44. Luo Z, Mishra A, Achkar A, Eichel J, Li S, Jodoin PM (2017) Non-local deep features for salient object detection. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 6593–6601
45. Chen S, Tan X, Wang BN, Hu X (2018) Reverse attention for salient object detection. In: Proceedings of ECCV, pp 234–250
46. Zeng Y, Zhuge Y, Lu H, Zhang L, Qian M, Yu Y (2019) Multi-source weak supervision for saliency detection. In: Proceedings of IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp 6074–6083
47. Deng Z, Hu X, Zhu L, Xu X, Qin J, Han G, Heng PA (2018) R3net: Recurrent residual refinement network for saliency detection. In: Proceedings of Int. Joint Conf. Artif. Intell. (AAAI), pp 684–690
48. Wang T, Borji A, Zhang L, Zhang P, Lu H (2017) A stagewise refinement model for detecting salient objects in images. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 4039–4048
49. Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks, [J]. J Mach Learn Res 9:249–256
50. He K, Zhang X, Ren S et al (2015) Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In: Proceedings of IEEE Int. Conf. Comput. Vis. (ICCV), pp 1026–1034
51. Liu Z, Xu J, Peng X et al (2018) Frequency-domain dynamic pruning for convolutional neural networks, [C]//Advances in Neural Information Processing Systems, pp 1043–1053
52. Wang X, Liang J (2016) Scalable compression of deep neural networks, [C]//Proceedings of the 24th ACM international conference on Multimedia, pp 511–515
53. Chen Z, Wang S, Wu D et al (2018) From data to knowledge: Deep learning model compression, transmission and communication, [C]//Proceedings of the 26th ACM international conference on Multimedia, pp 1625–1633
54. Bilal H, Kumar A, Yin B (2021) Pruning filters with L1-norm and capped L1-norm for CNN compression. Appl Intell 2:51
55. Singh P, Verma VK, Rai P, Namboodiri VP (2020) Acceleration of deep convolutional neural networks using adaptive filter pruning. IEEE J Sel Top Signal Process 14(4):838–847. <https://doi.org/10.1109/JSTSP.2020.2992390>
56. Chen Y, Guo B, Shen Y et al (2021) Using efficient group pseudo-3D network to learn spatio-temporal features[J]. SIVIP 15(2):361–369
57. Chen Y, Guo B, Shen Y et al (2021) Boundary graph convolutional network for temporal action detection[J]. Image Vis Comput 109:104144
58. Chen Y, Guo B, Shen Y, Wang W, Lu W, Suo X Capsule boundary network with 3D convolutional dynamic routing for temporal action detection. In: IEEE Transactions on Circuits and Systems for Video Technology. <https://doi.org/10.1109/TCSVT.2021.3104226>
59. Liu X, Wu L, Dai C, Chao H-C (2021) Compressing CNNs using multilevel filter pruning for the edge nodes of multimedia internet of things. IEEE Internet Things J 8(14):11041–11051. <https://doi.org/10.1109/JIOT.2021.3052016>
60. Dai C, Cheng H, Liu X (2020) A tucker decomposition based on adaptive genetic algorithm for efficient deep model compression. IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), pp 507–512. <https://doi.org/10.1109/HPCC-SmartCity-DSS50907.2020.00062>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Yaosen Chen received the B.E. degree from Southwest Jiaotong University, M.E. degree from University of Electronic Science and Technology of China, and Ph.D degree from Sichuan University in 2010, 2016, and 2021, respectively. His main research include computer vision and embedded system.



Renshuang Zhou is currently pursuing a master's degree in computer science at Sichuan University. He received his B.E. degree from Soochow University. His current research interests include image super-resolution and deep neural network compression.



Bing Guo received his B.S. degree in Computer Science from the Beijing Institute of Technology in China, and M.S. and Ph.D. degrees in Computer Science from the University of Electronic Science and Technology of China, China, in 1991, 1999, and 2002, respectively. He is currently a Professor in the College of Computer Science at the Sichuan University, China. His current research interests include embedded systems, big data management and block chain.



Yan Shen is currently a professor of Chengdu University of Information Technology. She received the Ph.D. from University of Electronic Science and Technology of China in 2004. Her research interests include Internet of things and big data analysis.



Xuming Wen received the B.E. degree and M.Sc. degree in electrical engineering from Xi'an Jiaotong University, Xi'an, China, in 1996 and 2003. His main research interests include data governance technology and natural language processing.



Wei Wang received the B.E. degree from Zhejiang University, China, the M.Sc. degree from the University of Electronic Science and Technology of China, China, and the D.Eng. degree from Sichuan University, China in 1997, 2004 and 2021, respectively. His main research interests include natural language processing and big data management.



Xinhua Suo is a doctoral student in Sichuan University. His current research interests mainly include Knowledge Graph (KG) Language Processing (NLP).